Q1. Reverse Engineering of a Black-Box Web Application Using Selenium

a) Application Reconnaissance and Reverse Engineering

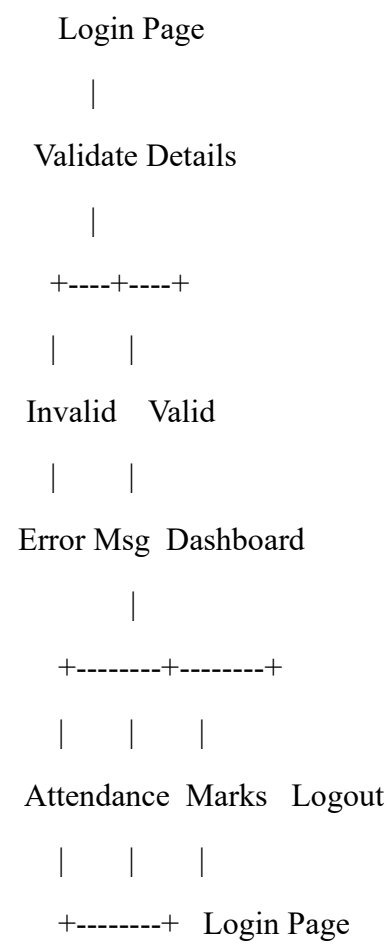
Aim: To study an existing student portal and identify its functions without accessing its internal code.

Tools: Selenium WebDriver, Chrome DevTools, and Excel.

Functional Requirements

1. The user should log in using a valid username and password.
2. The system should display an error for invalid login details.
3. After successful login, the dashboard should be displayed.
4. Students should be able to view attendance and marks.
5. The logout function should terminate the session.
6. Unauthorized users should not access internal pages.

Functional Model



Observations and Assumptions

- **Observation:** The login page accepts a username and password.
- **Observation:** The system displays an error when invalid credentials are rejected.
- **Assumption:** The application may lock an account after repeated failed attempts. This must be verified through testing.

b) Black-Box Test Design

- Black-box testing checks the application's functionality without examining its internal code.

Test ID	Input	Expected Result
TC01	Valid username and password	Login successful
TC02	Invalid password	Error message displayed
TC03	Invalid username	Login rejected
TC04	Empty username and password	Validation message displayed
TC05	Valid login followed by logout	Session terminated
TC06	Access dashboard without login	Access denied or redirected to login
TC07	Navigate backward after logout	Protected page should not be accessible
TC08	Random username and password	Invalid login rejected

Equivalence Class Partitioning: Divide inputs into valid and invalid categories.

Boundary Value Analysis: Test minimum and maximum permitted username and password lengths, once those restrictions are known.

c) Automation Using Selenium

Python Program

Install Selenium using:

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()
driver.get("http://localhost:5000/login")

wait = WebDriverWait(driver, 10)

driver.find_element(By.ID, "username").send_keys("student")
driver.find_element(By.ID, "password").send_keys("wrong")
driver.find_element(By.ID, "login").click()

try:
    wait.until(EC.visibility_of_element_located(
        (By.ID, "error")
    ))
    print("Invalid login test passed")
except Exception:
    driver.save_screenshot("failure.png")
    print("Test failed")

driver.quit()
```

Note: This example assumes the application uses the specified URL and element IDs. Replace them with the actual values. The application must be running before executing the script.

d) State Transition Testing

Logged Out

|

| Login

v

Login in Progress

|

| Valid Credentials

v

Logged In

|

| Logout / Timeout

v

Logged Out / Session Expired

Test valid login, invalid login, logout, session expiration, and browser navigation after logout.

e) Test Adequacy Evaluation

Test adequacy measures how effectively the test suite checks the application.

Formula:

Test Execution Percentage = (Executed Test Cases / Planned Test Cases) × 100

Pass Percentage = (Passed Test Cases / Executed Test Cases) × 100

For example, if 8 test cases are planned, 6 are executed, and 5 pass:

- Execution percentage = (6/8) × 100 = **75%**
- Pass percentage = (5/6) × 100 = **83.33%**

Conclusion: Selenium automation helps identify login, session, and access-control defects. Actual release readiness can be decided only after executing the tests and reviewing the results.

Q2. Reverse Engineering of a Java Application Using JaCoCo

a) Reverse Engineering and Logic Reconstruction

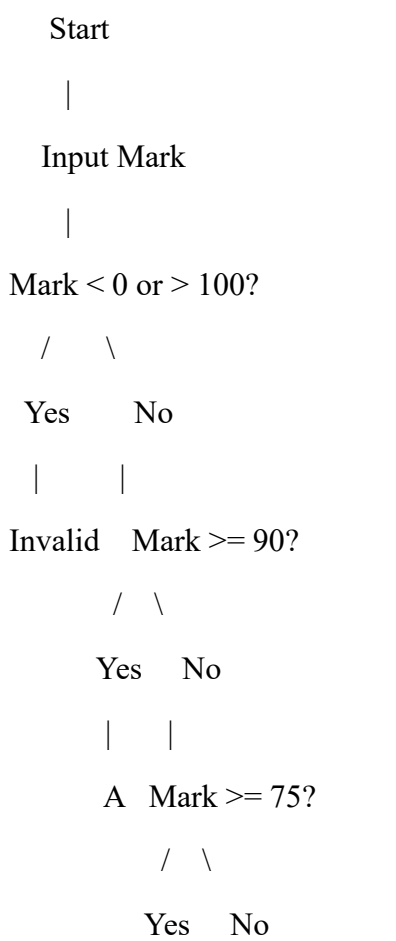
Aim: To understand the Java result calculator and test its grading logic.

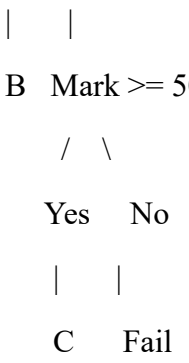
Program Logic

The program accepts a mark between 0 and 100 and returns a grade.

Mark Range	Output
Below 0	Invalid
0–49	Fail
50–74	C
75–89	B
90–100	A
Above 100	Invalid

Flowchart





b) Black-Box Test Cases

Test ID	Mark	Expected Output
TC01	-1	Invalid
TC02	0	Fail
TC03	49	Fail
TC04	50	C
TC05	74	C
TC06	75	B
TC07	89	B
TC08	90	A
TC09	100	A
TC10	101	Invalid

c) White-Box Testing

White-box testing examines the program's internal logic and execution paths.

Independent Paths

1. Invalid mark.
2. Valid mark with grade A.
3. Valid mark with grade B.
4. Valid mark with grade C.

5. Valid mark with Fail.

   Cyclomatic Complexity

   Formula:

$$V(G) = E - N + 2P$$

   Where:

- E = Number of edges.
- N = Number of nodes.
- P = Number of connected components.

   Using the conventional decision-node count for the four if/else if decisions:

$$V(G) = 4 + 1 = \boxed{5}$$

   Therefore, the program has a cyclomatic complexity of **5** under this CFG model.

   JUnit Test Program

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

public class ResultCalculatorTest {

  @Test
  void testGrades() {
    assertEquals("Invalid",
      ResultCalculator.calculateGrade(-1));
    assertEquals("Fail",
      ResultCalculator.calculateGrade(49));
    assertEquals("C",
      ResultCalculator.calculateGrade(50));
    assertEquals("B",
      ResultCalculator.calculateGrade(75));
    assertEquals("A",
```

```
            ResultCalculator.calculateGrade(90));
        assertEquals("Invalid",
            ResultCalculator.calculateGrade(101));
    }
}
```

Note: This program requires the JUnit 5 library and the provided ResultCalculator class.

d) JaCoCo Coverage Analysis

JaCoCo measures the code executed by the tests.

Steps:

1. Create a Maven-based Java project.
2. Add JUnit and JaCoCo dependencies.
3. Write test cases for all grade ranges.
4. Run the tests using Maven.
5. Generate the JaCoCo HTML report.
6. Check instruction, line, branch, and method coverage.
7. Add tests for any uncovered branches.

Types of Coverage

- **Statement coverage:** Checks whether statements execute.
- **Branch coverage:** Checks true and false decision outcomes.
- **Condition coverage:** Checks individual Boolean conditions.

e) Conclusion

Reverse engineering helps reconstruct the grading rules from the existing code. JUnit and JaCoCo help identify missing test cases and improve code coverage. However, high coverage alone cannot guarantee that all requirements are correct.

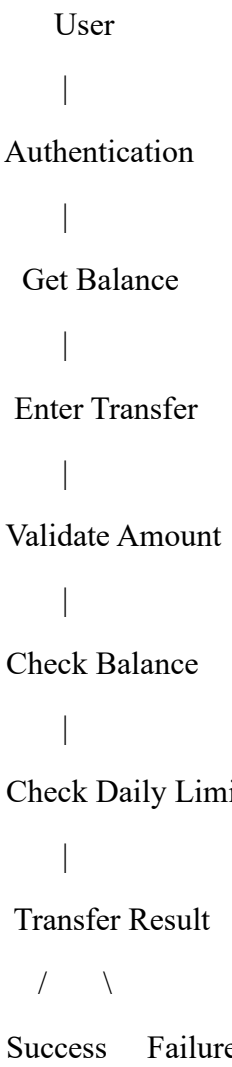
Q3. Reverse Engineering and API Testing of a Banking Application

a) Aim

To reverse engineer a banking REST API and test account balance retrieval and fund transfer operations using Postman.

Tools: Postman, browser developer tools, and Excel.

API Functional Model



Intended Transaction Rules

1. The transfer amount must be greater than zero.
2. The amount must not exceed the available balance.
3. The amount must not exceed the daily limit.
4. Unauthenticated users must not complete transfers.
5. Failed transactions must not incorrectly reduce the balance.

b) API Test Cases

Test ID	Input / Condition	Expected Result
TC01	Valid amount and authentication	Transfer successful
TC02	Amount = 0	Request rejected
TC03	Negative amount	Request rejected
TC04	Amount greater than balance	Transfer rejected
TC05	Amount greater than daily limit	Transfer rejected
TC06	Missing authentication	Unauthorized response
TC07	Missing amount field	Validation error
TC08	Amount supplied as text	Invalid input rejected
TC09	Invalid account identifier	Appropriate error response
TC10	Failed transaction	Balance remains unchanged

c) Decision Table

Authentication	Sufficient Balance	Within Daily Limit	Expected Result
Valid	Yes	Yes	Transfer allowed
Invalid	Yes	Yes	Reject
Valid	No	Yes	Reject
Valid	Yes	No	Reject
Invalid	No	No	Reject

d) State Transition Testing

Pending

|

+----> Successful

|

+----> Failed

|

+----> Cancelled

e) White-Box Testing

If source code is available, construct a Control Flow Graph and calculate cyclomatic complexity.

If source code is unavailable, internal paths and code coverage cannot be directly measured. Instead, use API responses, request validation, transaction outcomes, and balance comparisons.

f) Test Adequacy and Defect Reporting

A defect report should contain:

- Defect ID
- Test case ID
- Description
- Steps to reproduce
- Expected result
- Actual result
- Severity
- Status

Conclusion: API testing helps identify transaction validation, authentication, and balance-management defects. Testing should be performed only on an authorized training API or mock application.

Q4. Reverse Engineering a COTS Application

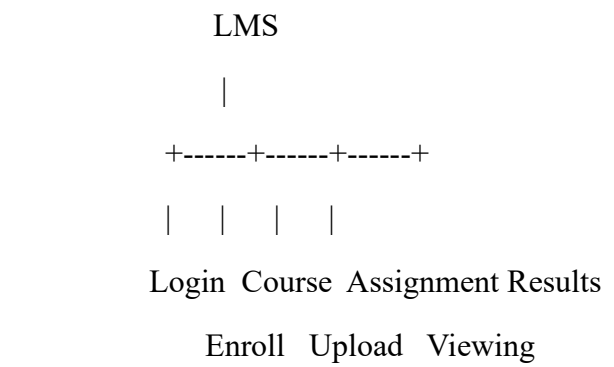
a) Aim

To test the functionality, compatibility, and performance of an existing Learning Management System (LMS) without accessing its source code.

COTS: Commercial Off-The-Shelf software is ready-made software developed for general use and purchased by an organization.

Tools: Selenium WebDriver, Apache JMeter, browser DevTools, and spreadsheets.

b) Functional Decomposition



Functional Test Cases

Test ID	Test Scenario	Expected Result
TC01	Valid login	Dashboard displayed
TC02	Invalid login	Error displayed
TC03	Enroll in available course	Enrollment successful
TC04	Upload valid assignment	File uploaded
TC05	Upload unsupported file type	File rejected
TC06	Upload file exceeding the limit	Validation error
TC07	View examination results	Results displayed
TC08	Access restricted page without login	Access denied

c) Compatibility Testing

Compatibility testing checks whether the application works correctly across supported environments.

Test combinations of:

- Different browsers.
- Different operating systems.
- Desktop and mobile devices.
- Different screen sizes.

Example Compatibility Matrix

Browser	Desktop	Mobile
Chrome	Test	Test
Firefox	Test	Test
Edge	Test	Test

d) Performance Testing Using JMeter

Steps:

1. Open Apache JMeter.
2. Create a Test Plan.
3. Add a Thread Group.
4. Configure the number of virtual users.
5. Add an HTTP Request sampler for an authorized test endpoint.
6. Add listeners to view results.
7. Run the test.
8. Record response time, throughput, and error rate.

Performance Measurements

- Response time: Time taken to respond to a request.
- Throughput: Number of requests processed per unit of time.
- Error rate: Percentage of requests that fail.

Compare the results against predefined performance requirements.

e) Limitations of COTS Testing

Since the source code is unavailable:

1. Internal code paths cannot be examined directly.
2. Control Flow Graph construction is not possible without implementation details.
3. JaCoCo-style source-code coverage cannot be directly measured.
4. Testing must rely on observable behavior and available documentation.

Alternative approaches include black-box testing, API testing, compatibility testing, and acceptance testing.

f) Conclusion

COTS testing helps determine whether the LMS meets the university's requirements. Based on test results and acceptance criteria, the application can be accepted, conditionally accepted, or rejected.

Q5. Reverse Engineering a Python Application Using Pytest and Coverage.py

a) Aim

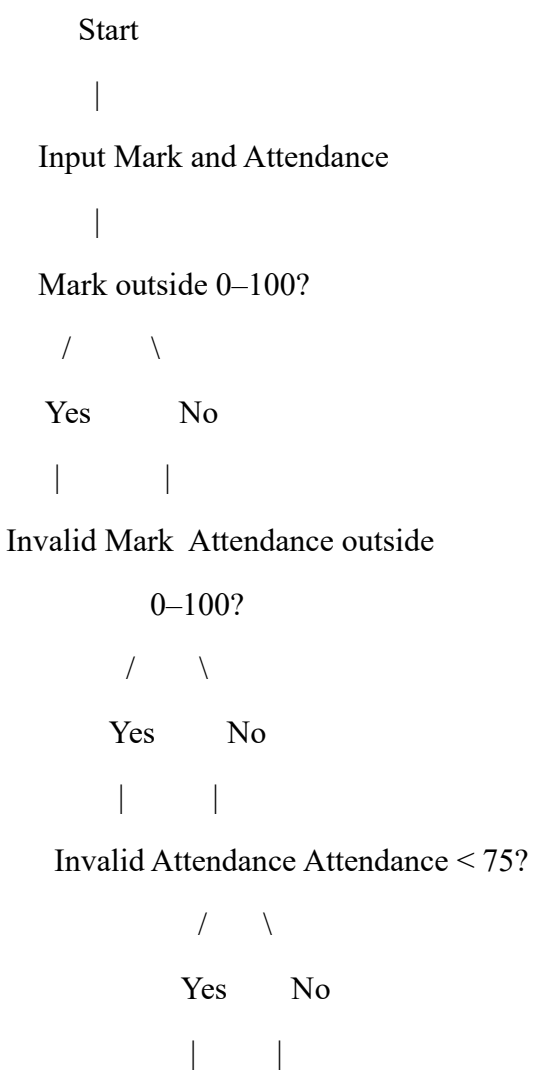
To reconstruct the logic of a Python result-validation program and test it using pytest and Coverage.py.

Program Logic

The program accepts marks and attendance as inputs.

1. Marks outside 0–100 return Invalid Mark.
2. Attendance outside 0–100 returns Invalid Attendance.
3. Attendance below 75 returns Attendance Shortage.
4. Attendance of at least 75 and marks of at least 50 return Pass.
5. Otherwise, the result is Fail.

b) Flowchart



Attendance Mark >= 50?

Shortage / \

Yes No

| |

Pass Fail

c) Black-Box Test Cases

Test ID	Mark	Attendance	Expected Output
TC01	-1	80	Invalid Mark
TC02	50	101	Invalid Attendance
TC03	40	70	Attendance Shortage
TC04	50	75	Pass
TC05	49	75	Fail
TC06	100	100	Pass
TC07	0	75	Fail
TC08	101	80	Invalid Mark
TC09	50	0	Attendance Shortage

d) Pytest Program

```
def validate_result(mark, attendance):
    if mark < 0 or mark > 100:
        return "Invalid Mark"

    if attendance < 0 or attendance > 100:
        return "Invalid Attendance"

    if attendance < 75:
```

```
        return "Attendance Shortage"

    if mark >= 50:
        return "Pass"

    return "Fail"
```

Create another file named test_result.py:

```
from result import validate_result

def test_invalid_mark():
    assert validate_result(-1, 80) == "Invalid Mark"

def test_invalid_attendance():
    assert validate_result(50, 101) == "Invalid Attendance"

def test_shortage():
    assert validate_result(40, 70) == "Attendance Shortage"

def test_pass():
    assert validate_result(50, 75) == "Pass"

def test_fail():
    assert validate_result(49, 75) == "Fail"
```

Run the Tests

Install the required packages:

```
pip install pytest coverage
```

Run the tests:

```
pytest -v
```

Measure statement coverage:

coverage run -m pytest

coverage report -m

To measure branch coverage, run:

coverage run --branch -m pytest

coverage report -m

e) Cyclomatic Complexity

The program contains four main decision points:

1. Mark validation.
2. Attendance validation.
3. Attendance eligibility.
4. Pass/fail condition.

Using the conventional decision-node formula:

$$V(G) = \text{Number of decisions} + 1$$
$$V(G) = 4 + 1 = \boxed{5}$$

Therefore, the cyclomatic complexity is 5 under this model.

f) Mutation Testing

Mutation testing checks whether the test cases can detect small changes in the program.

Example 1: Change attendance < 75 to attendance <= 75.

- Test input: mark = 50, attendance = 75.
- Original output: Pass.
- Modified output: Attendance Shortage.
- The test should fail, detecting the change.

Example 2: Change mark >= 50 to mark > 50.

- Test input: mark = 50, attendance = 75.
- Original output: Pass.
- Modified output: Fail.
- The test should fail, detecting the change.

g) Conclusion

Reverse engineering helps identify the program's decision logic and missing requirements. Pytest checks the expected results, while Coverage.py identifies untested statements and branches. Mutation testing further evaluates whether the test cases can detect incorrect logic.